

Unit A1: Computer Fundamentals (HL Extension)

Sub-topics: A1.1 to A1.3 (HL Specifics)

1. Syllabus Scope & Learning Outcomes

These topics are for Higher Level students only.

Syllabus Point	Command Term	Student Expectation (The "Do")
A1.1.3	Explain	Explain differences between CPU (few powerful cores, serial tasks) and GPU (thousands of smaller cores, parallel tasks).
A1.1.6	Describe	Describe Pipelining (fetch/decode/execute overlap) in multi-core architectures.
A1.3.5	Explain	Explain OS role in multitasking: Resource contention, Deadlock, and Starvation.
A1.3.6	Describe	Describe Control System components: Sensor -> ADC -> Controller -> DAC -> Actuator.
A1.3.7	Explain	Explain Open-Loop vs Closed-Loop (Feedback) systems in real contexts (e.g., AC, Cruise Control).
A1.4.1	Evaluate	Evaluate Interpreters vs Compilers vs JIT

		(Just-In-Time) regarding speed, portability, and error detection.
--	--	---

2. Key Terminology & Definitions

- **Pipelining:** A technique where the CPU fetches the *next* instruction while decoding the *current* one, maximizing efficiency.
- **Parallel Processing:** Using multiple cores (or a GPU) to execute independent tasks simultaneously.
- **Deadlock:** A situation where two processes are each waiting for the other to release a resource, causing a freeze.
- **Sensor:** Input device measuring physical data (Temp, Light, Motion).
- **Actuator:** Output device producing physical movement/action (Motor, Valve, Heater).
- **ADC / DAC:** Analogue-to-Digital Converter / Digital-to-Analogue Converter.
- **Feedback Loop:** The output of a system is measured and fed back as input to adjust the process (Closed-Loop).
- **Compiler:** Translates entire source code to machine code *before* execution.
- **Interpreter:** Translates and executes source code line-by-line.

3. Core Content & Exam Actions

A1.1.3 & A1.1.6: Advanced Architecture

Theory:

- **CPU vs GPU:**
 - **CPU:** Low latency, complex logic, branch prediction. Good for OS, serial code.
 - **GPU:** High throughput, SIMD (Single Instruction Multiple Data). Good for graphics, ML, matrix math.
- **Pipelining:**
 - Concept: Like a factory assembly line.
 - Stages: Fetch -> Decode -> Execute -> Write Back.
 - Benefit: Increases throughput (instructions completed per clock cycle).

Student Exam Action:

- **Explanation:** Explain why a GPU is better for rendering 3D graphics than a CPU (Parallel processing of pixels).

A1.3.5: Resource Management

Theory:

- **Multitasking:** The OS slices CPU time to create the illusion of parallel execution on a single core.

- **Problems:**
 - **Deadlock:** Process A holds Resource 1, needs Resource 2. Process B holds Resource 2, needs Resource 1.
 - **Starvation:** Low-priority process never gets CPU time (solved by "Aging" priority).

Student Exam Action:

- **Scenario:** Given a scenario of a frozen printer queue, identify if it is a Deadlock or Starvation issue.

A1.3.6 & A1.3.7: Control Systems

Theory:

- **Open-Loop:** No feedback. Action happens for a set time (e.g., Toaster, basic Sprinkler).
- **Closed-Loop:** Feedback used. System self-corrects (e.g., Thermostat, Cruise Control).
- **Process:** Sensor -> ADC -> Microprocessor (compares to pre-set value) -> DAC -> Actuator.

Student Exam Action:

- **Diagram:** Draw the block diagram for a Central Heating System (Sensor -> Processor -> Heater -> Output -> Sensor).

A1.4.1: Translators (Compilers vs Interpreters)

Theory:

- **Compiler:**
 - Pros: Fast execution, code is hidden (binary), optimized.
 - Cons: Slow compilation, platform dependent.
- **Interpreter:**
 - Pros: Starts immediately, easy debugging, platform independent source.
 - Cons: Slower execution, source code visible.
- **Java:** Hybrid. Compiles to **Bytecode** (intermediate), then interpreted by JVM (Java Virtual Machine).

Student Exam Action:

- **Evaluation:** Recommend a translator for a developer releasing a closed-source mobile game (Compiler - for speed and code protection).

4. Common Pitfalls & Examiner Tips

- **Pitfall (Control Systems):** Forgetting the **ADC/DAC** conversion. Microprocessors work in Digital; the real world (Temperature, Sound) is Analogue. You *must* mention the conversion.
- **Pitfall (Feedback):** Confusing Open and Closed loops. Ask: "Does the system *know* if it finished the job?" If yes = Closed. If no = Open.
- **Tip (Translators):** "Java uses both" is a standard exam trick. It compiles to Bytecode,

then interprets Bytecode. This allows "Write Once, Run Anywhere."